# AULA 14: FUNÇÕES

Profª. Drª. Tatiele Bolson Moro

# Visão geral

- Para fazer um programa  grande e complexo temos que dividi-lo em blocos, ou seja, fazer unidades autônomas para realizar determinadas tarefas. Para esses blocos damos o nome de funções, a forma geral de uma função é:

```
/*Sintaxe de uma função*/
Tipo_de_retorno nomedafuncao(parâmetros)
{
     Comandos;
}
```

- O tipo_de_retorno é o tipo do  dado que a função vai retornar que se não for especificado, retornará um tipo int, ou seja, o default é o tipo int.
- Os parâmetros são uma lista de valores que a função recebe, ou seja, é a entrada da função e  tem a seguinte forma:

```
(parâmetros) = (Tipo nome1, tipo nome2, tipo nome3,..., tipo nomeN)
```

# Um exemplo simples de um função:
# Função que calcula o quadrado de um número

```c
#include <stdio.h>
#include <stdlib.h>
int quadrado(int a)
{
  return (a*a);
}
int main ()
{
  int numero;
  printf ("Entre com um numero: ");
  scanf ("%d",&numero);
  numero = quadrado(numero); /*chamada da função */
  printf ("\n\nO seu quadrado vale: %d\n", numero);
  system("pause");
  return 0;
}
```

```
PS C:\Users\Tatiele> cd 'c:\U
PS C:\Users\Tatiele\Desktop\U
Entre com um numero: 5


O seu quadrado vale: 25
Pressione qualquer tecla para
```

# Biblioteca <**stdlib.h**>

- No cabeçalho **stdlib.h** estão localizadas as funções responsáveis pela manipulação da alocação de memória, e da desalocação também, bem como funções para converter números que estão representados em strings para algum tipo de dado responsável por representar números, como, por exemplo, double, entre outras funcionalidades úteis, como funções de algoritmos de ordenação.

- https://petbcc.ufscar.br/stdlib/

- Toda função necessita ser chamada. Podemos perceber que no exemplo a chamada da função é:

```
numero = quadrado(numero);
```

- O dado de entrada da função é o argumento da chamada à função. No exemplo o dado de entrada da função quadrado é `numero`.
- O parâmetro da função quadrado é `int a`, onde `a` recebe o valor de entrada, ou seja, os parâmetros recebem os valores dos argumentos da chamada da função.
- Percebemos também que o tipo da variável `numero` é igual ao da variável `a`. Esta relação deve ser satisfeita sempre.

- Uma função é processada até chegar no seu valor de retorno (return).
- O valor de retorno deve ser compatível ao tipo de retorno declarado na função.
- No exemplo, a função `quadrado` retorna o valor `a*a`.

# Exemplo

```c
#include <stdio.h>
#include<stdlib.h>
int EhPar (int b)
{
  if (b%2)  /* Verifica se a e divisivel por dois */
    return 0;   /* Retorna 0 se nao for divisivel */
  else
    return 1;    /* Retorna 1 se for divisivel */
}
int main ()
{
  int numero;
  printf ("Entre com numero: ");
  scanf ("%d",&numero);
  if (EhPar(numero))
    printf ("\n\nO numero e par.\n");
  else
    printf ("\n\nO numero e impar.\n");
  system("pause");
  return 0;
}
```

Lembrar!
1 é verdadeiro
0 é falso

```
PS C:\Users\Tatiele> cd 'c:\Us
PS C:\Users\Tatiele\Desktop\UN
Entre com numero: 5


O numero e impar.
Pressione qualquer tecla para
```

# Protótipo

- Protótipos são as declarações das funções. Nos exemplos anteriores, **as funções** foram escritas **antes da função principal** (main) com o objetivo de informar ao compilador com antecedência quais eram os tipos de retorno e quais eram os parâmetros das funções para que este pudesse compilar o programa corretamente.

- Os protótipos são colocados antes da função principal, para **informar os tipos de retorno e parâmetros**, desta forma, a função pode ser escrita depois da função main, os protótipos tem a seguinte forma geral:

```
tipo_de_retorno nome_da_função (parâmetros);
```

# Exemplo 1:

```c
#include <stdio.h>
#include <stdlib.h>
int quadrado(int a);  //Protótipo da função
int main ()
{
  int numero;
  printf ("Entre com um numero: ");
  scanf ("%d",&numero);
  numero = quadrado(numero); /*chamada da função */
  printf ("\n\nO seu quadrado vale: %d\n",numero);
  system("pause");
  return 0;
}
int quadrado(int a)
{
  return (a*a);
}
```

Onde o tipo_de_retorno nome_da_funcao parâmetros devem ser os mesmos usados na função. A título de exemplo, reescrevemos os dois exemplos da seção anterior usando protótipos:

```
PS C:\Users\Tatiele> cd "c:\
Entre com um numero: 7


O seu quadrado vale: 49
Pressione qualquer tecla par
```

# Exemplo 2:

```
#include <stdio.h>
#include<stdlib.h>
int EhPar (int b)
int main ()
{
int numero;
printf ("Entre com numero: ");
scanf ("%d",&numero);
if (EhPar(numero))
  printf ("\n\nO numero e par.\n");
else
  printf ("\n\nO numero e impar.\n");
system("pause");
return 0;
}
int EhPar (int b)
{
  if (b%2)  /* Verifica se a e divisivel por dois */
    return 0;   /* Retorna 0 se nao for divisivel */
  else
    return 1;   /* Retorna 1 se for divisivel */
}
```

```
PS C:\Users\Tatiele> cd "c
Entre com um numero: 3


O seu quadrado vale: 9
Pressione qualquer tecla p
```

# Função sem retorno

- Se quisermos usar uma função, mas ela não retorna nenhum valor, declaramos o tipo de retorno da função como sendo void. Protótipo:

```
void nome_da_funcao (parâmetros);
```

- Também podemos ter funções que não tem entrada. Assim seu protótipo fica da seguinte forma:

```
Tipo_de_retorno nome_da_funcao (void);
```

- Ou ainda funções que não tem entrada e não retornam nenhum valor.

```
void nome_da_funcao (void);
```

# Exemplo 3: função void-void

```c
#include <stdio.h>
#include <stdlib.h>
void Mens (void);
int main ()
{
  Mens();
  printf ("\tRepita!:\n");
  Mens();
  system("pause");
  return 0;
}
void Mens (void)
{
  printf ("PET Elétrica! \n");
}
```

```
PS C:\Users\Tatiele> cd "c:
Entre com um numero: 10


O seu quadrado vale: 100
Pressione qualquer tecla pa
```

# Vetores como parâmetros de funções

- Além de variáveis, podemos também passar vetores para funções, algumas observações devem ser feitas a respeito disso.

- Ao passarmos variáveis para funções, estamos passando apenas o seu valor, ou seja, seu valor é processado, modificado e retornado.

- Quando passamos vetores, estamos, na realidade, passando o endereço do primeiro elemento do vetor.

- Assim, iremos processar os valores dos vetores, mas não os retornaremos, pois eles foram modificados dentro de seus endereços.

- Para vetores, temos o seguinte protótipo:

```
void func (int vetx[50]);
void func (int vetx[]);
```

- O mesmo ocorre com vetores bidimensionais (matrizes). Só temos uma observação a ser feita. Ao passarmos vetores bidimensionais, devemos sempre especificar o tamanho da “primeira dimensão”. Isto é feito para reconhecer o endereço corretamente das “duas dimensões”.
- Para matrizes, temos o seguinte protótipo:

```
void func (int matrx[50][]);
```

# Recursividade

- Uma poderosa característica de muitas linguagens de programação, incluindo o C, é o fato de que uma função pode chamar a si mesma!

- Quando isso ocorre, dizemos que a função é **recursiva**.

- Um  exemplo clássico de função recursiva é para o cálculo de fatorial.

# Exemplo Fatorial

```c
#include <stdio.h>
#include <stdlib.h>

int fatorial(int n) {
    if (n == 0)
        return 1;
    else
        return n * fatorial(n - 1); // Chama a própria função
}

/* Enquanto n não for zero, a função fatorial chama a si mesma
sempre com um valor de n menor que o anterior. O critério de
parada é n = 0. */
int main() {
    int n;
    printf("\n\nDigite um valor para n: ");
    scanf("%d", &n);
    printf("\nO fatorial de %d eh %d\n", n, fatorial(n));
    system("pause");
}
```

```
PS C:\Users\Tatiele> cd "C:\Users\Ta


Digite um valor para n: 3

O fatorial de 3 eh 6
Pressione qualquer tecla para contin
```

- Algumas observações relevantes devem ser feitas a respeito do uso da recursividade:
  - Deve-se tomar bastante cuidado ao usar recursividade, tenha muita atenção no critério de parada para evitar erros de lógica e que a função “se chame” infinitas vezes.
  - A cada vez que o computador faz uma chamada a uma função, é consumida uma parcela da memória. Assim, o uso incorreto da recursividade pode esgotar rapidamente a memória do computador.
  - A recursividade é evitada, sempre que possível.

# Escopo de variáveis

- Variáveis locais:  Este tipo de variável só tem validade dentro do bloco no qual foram declaradas.
  - Bloco, é um conjunto de instruções dentro de chaves { BLOCO }.
- Dessa forma, podemos declarar variáveis dentro de um for.
- A característica que torna as variáveis locais tão importantes é justamente a de serem exclusivas do bloco.
- Uma variável de nome **A** pode estar em quantos blocos desejarmos, ela não causará conflito!

# Exemplo 4:

```
/*Variáveis locais*/
func1 (...)
{
  int abc,x;
  ...
}
func (...)
{
        int abc;
        ...
}
void main ()
{
  int a,x,y;
  for (...)
  {
        float a,b,c;
        ...
  }
  ...
}
```

- Variáveis globais:  Este tipo de variável é declarada fora de todas as funções do programa.
- Pode ser acessada e alterada por todas as funções do programa.
- Uma função pode ter uma variável local com o mesmo nome de uma variável global.
- Quando isto ocorrer, a função dará prioridade à variável local.

```
/*Variáveis globais e locais*/
int z,k;
func1 (...)
{
      int x,y;
      ...
}
func2 (...)
{
      int x,y,z;
      ...
      z=10;
      ...
}
main ()
{
      int count;
      ...
}
```

- Variáveis formais:  Este tipo de variável é a declarada como sendo  à entrada de uma função.

- Assim, seu escopo é de variável local da função.